

# CS3014 - Estructura de Datos Avanzados

## Montículos

### Semana 1

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

12 de agosto de 2026

## ¿Qué aprenderemos hoy?

---



## ¿Qué aprenderemos hoy?

---



### ► Montículo Binario

## ¿Qué aprenderemos hoy?

---



- ▶ Montículo Binario
- ▶ Montículo Binomial

# Montículo Binario

---

## Montículo Binario — Introducción

---



### Definición

Un **montículo binario** (*binary heap*, o **montículo máximo**) es un arreglo que representa un árbol binario *casi completo*, en el cual cada nodo satisface la **propiedad de montículo**: el valor de un nodo es mayor o igual que el de sus hijos.

## Montículo Binario — Introducción



### Definición

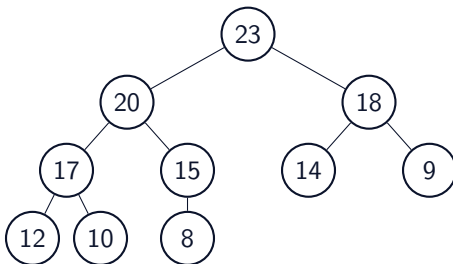
Un **montículo binario** (*binary heap*, o **montículo máximo**) es un arreglo que representa un árbol binario *casi completo*, en el cual cada nodo satisface la **propiedad de montículo**: el valor de un nodo es mayor o igual que el de sus hijos.

### Indexación (1-indexado)

Para un nodo en la posición  $i$ :

$$\text{padre}(i) = \left\lfloor \frac{i}{2} \right\rfloor, \quad \text{izq}(i) = 2i, \quad \text{der}(i) = 2i + 1.$$

## Un montículo máximo de ejemplo



| índice | 1  | 2  | 3  | 4  | 5  | 6  | 7 | 8  | 9  | 10 |
|--------|----|----|----|----|----|----|---|----|----|----|
| valor  | 23 | 20 | 18 | 17 | 15 | 14 | 9 | 12 | 10 | 8  |

## Max-Heapify

---



### Idea

Si los subárboles con raíz en  $\text{izq}(i)$  y  $\text{der}(i)$  ya son montículos máximos, pero  $A[i]$  podría ser menor que alguno de sus hijos, **MAX-HEAPIFY** “hunde” a  $A[i]$  hasta restaurar la propiedad de montículo en todo el subárbol con raíz  $i$ .

## Max-Heapify

---



---

### Algoritmo 1: MAX-HEAPIFY( $A, i, n$ )

---

$l \leftarrow 2i, \quad r \leftarrow 2i + 1 ;$

$m \leftarrow i ;$

**si**  $l \leq n$  **y**  $A[l] > A[m]$  **entonces**

$m \leftarrow l$

**si**  $r \leq n$  **y**  $A[r] > A[m]$  **entonces**

$m \leftarrow r$

**si**  $m \neq i$  **entonces**

    intercambiar  $A[i]$  y  $A[m] ;$

    MAX-HEAPIFY( $A, m, n$ )

---

## Max-Heapify — ejemplo trabajado

---



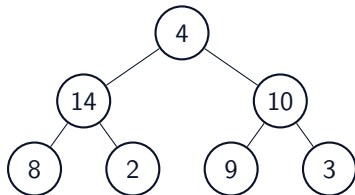
Antes de llamar  $\text{MAX-HEAPIFY}(A, 1, 7)$

$A = [4, 14, 10, 8, 2, 9, 3]$ : los subárboles con raíz 2 y 3 ya son montículos máximos válidos, pero la raíz  $A[1] = 4$  es menor que ambos hijos.

## Max-Heapify — ejemplo trabajado

Antes de llamar  $\text{MAX-HEAPIFY}(A, 1, 7)$

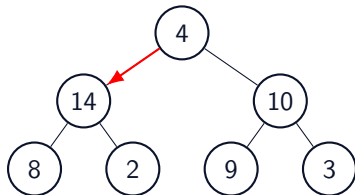
$A = [4, 14, 10, 8, 2, 9, 3]$ : los subárboles con raíz 2 y 3 ya son montículos máximos válidos, pero la raíz  $A[1] = 4$  es menor que ambos hijos.



## Max-Heapify — ejemplo trabajado

Antes de llamar  $\text{MAX-HEAPIFY}(A, 1, 7)$

$A = [4, 14, 10, 8, 2, 9, 3]$ : los subárboles con raíz 2 y 3 ya son montículos máximos válidos, pero la raíz  $A[1] = 4$  es menor que ambos hijos.

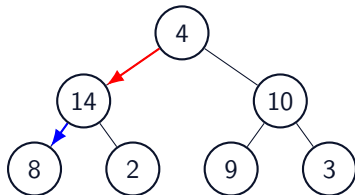


Paso 1: mayor hijo de la raíz es 14, se intercambia con 4.

## Max-Heapify — ejemplo trabajado

Antes de llamar  $\text{MAX-HEAPIFY}(A, 1, 7)$

$A = [4, 14, 10, 8, 2, 9, 3]$ : los subárboles con raíz 2 y 3 ya son montículos máximos válidos, pero la raíz  $A[1] = 4$  es menor que ambos hijos.



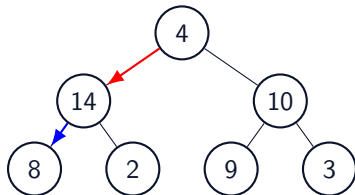
Paso 1: mayor hijo de la raíz es 14, se intercambia con 4.

Paso 2: en la posición donde quedó el 4, su mayor hijo es 8, se intercambia de nuevo.

## Max-Heapify — ejemplo trabajado

Antes de llamar  $\text{MAX-HEAPIFY}(A, 1, 7)$

$A = [4, 14, 10, 8, 2, 9, 3]$ : los subárboles con raíz 2 y 3 ya son montículos máximos válidos, pero la raíz  $A[1] = 4$  es menor que ambos hijos.



Paso 1: mayor hijo de la raíz es 14, se intercambia con 4.

Paso 2: en la posición donde quedó el 4, su mayor hijo es 8, se intercambia de nuevo.

**Resultado:**  $A = [14, 8, 10, 4, 2, 9, 3]$  — ya es un montículo máximo válido.

## Complejidad de Max-Heapify: ¿qué cuenta?



### Observación clave

MAX-HEAPIFY hace  $O(1)$  de trabajo (dos comparaciones, y quizás un intercambio) y luego se llama a sí misma **a lo mucho una vez**, sobre *uno* de los dos hijos — nunca sobre ambos.

## Complejidad de Max-Heapify: ¿qué cuenta?



### Observación clave

MAX-HEAPIFY hace  $O(1)$  de trabajo (dos comparaciones, y quizás un intercambio) y luego se llama a sí misma **a lo mucho una vez**, sobre *uno* de los dos hijos — nunca sobre ambos.

¿Qué determina el costo total, entonces?

Cada llamada avanza un nivel más abajo en el árbol, siguiendo un único camino (como en el ejemplo anterior: raíz  $\rightarrow$  hijo izquierdo  $\rightarrow$  su hijo izquierdo). El número de llamadas recursivas está acotado por la **altura** del subárbol.

## Complejidad de Max-Heapify: la altura

---



¿Cuál es la altura de un heap de  $n$  elementos?

Un árbol binario casi completo con  $n$  nodos tiene altura  $h = \lfloor \lg n \rfloor$ : el número de nodos (casi) se duplica en cada nivel, así que  $2^h \leq n < 2^{h+1}$ .

## Complejidad de Max-Heapify: la altura



¿Cuál es la altura de un heap de  $n$  elementos?

Un árbol binario casi completo con  $n$  nodos tiene altura  $h = \lfloor \lg n \rfloor$ : el número de nodos (casi) se duplica en cada nivel, así que  $2^h \leq n < 2^{h+1}$ .

### Conclusión

$O(1)$  de trabajo por nivel, y a lo mucho  $O(\lg n)$  niveles:  $T(n) = O(\lg n)$ .

## Build-Max-Heap

---



---

### Algoritmo 2: BUILD-MAX-HEAP( $A, n$ )

---

para  $i \leftarrow \lfloor n/2 \rfloor$  a 1 hacer  
     $\text{MAX-HEAPIFY}(A, i, n)$  ;

---

## Build-Max-Heap



---

### Algoritmo 3: BUILD-MAX-HEAP( $A, n$ )

---

para  $i \leftarrow \lfloor n/2 \rfloor$  a 1 hacer  
     $\text{MAX-HEAPIFY}(A, i, n)$  ;

---

¿Por qué basta empezar en  $\lfloor n/2 \rfloor$ ?

Toda posición  $i > \lfloor n/2 \rfloor$  es una hoja (no tiene hijos), así que ya es trivialmente un montículo máximo de un solo nodo.

¿Build-Max-Heap es  $\Theta(n \lg n)$  o algo mejor?

---



¡Piénsalo!

Una cota ingenua:  $\lfloor n/2 \rfloor$  llamadas a MAX-HEAPIFY, cada una  $O(\lg n)$  (por lo que acabamos de ver)  $\Rightarrow O(n \lg n)$  en total. ¿Es esta cota **ajustada**?

¿Build-Max-Heap es  $\Theta(n \lg n)$  o algo mejor?

---

¡Piénsalo!

Una cota ingenua:  $\lfloor n/2 \rfloor$  llamadas a MAX-HEAPIFY, cada una  $O(\lg n)$  (por lo que acabamos de ver)  $\Rightarrow O(n \lg n)$  en total. ¿Es esta cota **ajustada**?

### Lema

En un heap de  $n$  elementos, la cantidad de nodos a altura  $h$  es a lo mucho  $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ .

¿Build-Max-Heap es  $\Theta(n \lg n)$  o algo mejor?

¡Piénsalo!

Una cota ingenua:  $\lfloor n/2 \rfloor$  llamadas a MAX-HEAPIFY, cada una  $O(\lg n)$  (por lo que acabamos de ver)  $\Rightarrow O(n \lg n)$  en total. ¿Es esta cota **ajustada**?

### Lema

En un heap de  $n$  elementos, la cantidad de nodos a altura  $h$  es a lo mucho  $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ .

### Intuición

La mitad de los nodos son hojas (altura 0); un cuarto tiene altura a lo mucho 1; un octavo, altura a lo mucho 2; etc. La **mayoría** de las llamadas a MAX-HEAPIFY ocurren cerca de las hojas, donde “hunden” muy poco.

## Sumando el costo real sobre todos los niveles



### Costo total

Como MAX-HEAPIFY en un nodo de altura  $h$  cuesta  $O(h)$ :

$$T(n) \leq \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right).$$

## Sumando el costo real sobre todos los niveles



### Costo total

Como MAX-HEAPIFY en un nodo de altura  $h$  cuesta  $O(h)$ :

$$T(n) \leq \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right).$$

### Un hecho útil

La serie  $\sum_{h=0}^{\infty} \frac{h}{2^h}$  **converge** a 2 (es una serie geométrica derivada término a término).

## Sumando el costo real sobre todos los niveles

---



### Conclusión

$T(n) = O(n \cdot 2) = O(n)$ . BUILD-MAX-HEAP es  $\Theta(n)$ : ¡la cota ingenua de  $O(n \lg n)$  **no** era ajustada!

# Montículo Binomial

---

# Árboles Binomiales

---



## Definición (recursiva)

El árbol binomial  $B_0$  es un solo nodo. El árbol binomial  $B_k$  se construye uniendo dos árboles  $B_{k-1}$ : la raíz de uno se convierte en el hijo **más a la izquierda** de la raíz del otro.

# Árboles Binomiales



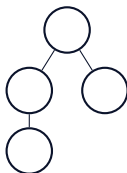
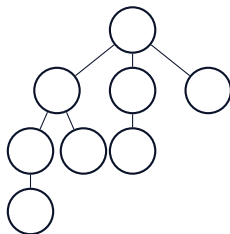
## Definición (recursiva)

El árbol binomial  $B_0$  es un solo nodo. El árbol binomial  $B_k$  se construye uniendo dos árboles  $B_{k-1}$ : la raíz de uno se convierte en el hijo **más a la izquierda** de la raíz del otro.

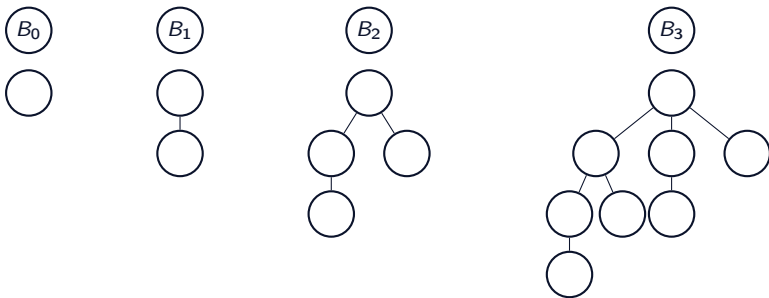
## Propiedades de $B_k$

- ▶ Tiene  $2^k$  nodos.
- ▶ Tiene altura  $k$ .
- ▶ La raíz tiene grado (número de hijos) igual a  $k$ .
- ▶ Tiene exactamente  $\binom{k}{i}$  nodos a profundidad  $i$  — de ahí el nombre *binomial*.

## Árboles Binomiales $B_0, B_1, B_2, B_3$

 $B_0$  $B_1$  $B_2$  $B_3$ 

# Árboles Binomiales $B_0, B_1, B_2, B_3$



$B_k$  se forma uniendo dos  $B_{k-1}$ : por eso la raíz de  $B_k$  tiene como hijos las raíces de  $B_0, B_1, \dots, B_{k-1}$ .

# Montículo Binomial

---



## Estructura

Un **montículo binomial** es una colección de árboles binomiales, a lo mucho uno de cada orden  $B_k$  — igual que los bits 1 en la representación binaria de  $n$ .

# Montículo Binomial



## Estructura

Un **montículo binomial** es una colección de árboles binomiales, a lo mucho uno de cada orden  $B_k$  — igual que los bits 1 en la representación binaria de  $n$ .

¿Por qué esto ayuda?

UNION de dos montículos binomiales es análogo a **sumar en binario**: se enlazan árboles del mismo orden (como un acarreo), en  $O(\lg n)$ .

## Comparando las tres estructuras

---



| Operación | Binario | Binomial | Fibonacci |
|-----------|---------|----------|-----------|
|-----------|---------|----------|-----------|

---

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras

---



| Operación | Binario         | Binomial   | Fibonacci   |
|-----------|-----------------|------------|-------------|
| insert    | $\Theta(\lg n)$ | $O(\lg n)$ | $\Theta(1)$ |

---

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras

---



| Operación | Binario         | Binomial   | Fibonacci   |
|-----------|-----------------|------------|-------------|
| insert    | $\Theta(\lg n)$ | $O(\lg n)$ | $\Theta(1)$ |
| find-min  | $\Theta(1)$     | $O(\lg n)$ | $\Theta(1)$ |

---

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras

---



| Operación   | Binario         | Binomial        | Fibonacci   |
|-------------|-----------------|-----------------|-------------|
| insert      | $\Theta(\lg n)$ | $O(\lg n)$      | $\Theta(1)$ |
| find-min    | $\Theta(1)$     | $O(\lg n)$      | $\Theta(1)$ |
| extract-min | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |

---

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras

---



| Operación   | Binario         | Binomial        | Fibonacci   |
|-------------|-----------------|-----------------|-------------|
| insert      | $\Theta(\lg n)$ | $O(\lg n)$      | $\Theta(1)$ |
| find-min    | $\Theta(1)$     | $O(\lg n)$      | $\Theta(1)$ |
| extract-min | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |
| union       | $\Theta(n)$     | $O(\lg n)$      | $\Theta(1)$ |

---

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras



| Operación    | Binario         | Binomial        | Fibonacci   |
|--------------|-----------------|-----------------|-------------|
| insert       | $\Theta(\lg n)$ | $O(\lg n)$      | $\Theta(1)$ |
| find-min     | $\Theta(1)$     | $O(\lg n)$      | $\Theta(1)$ |
| extract-min  | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |
| union        | $\Theta(n)$     | $O(\lg n)$      | $\Theta(1)$ |
| decrease-key | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $\Theta(1)$ |

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras



| Operación    | Binario         | Binomial        | Fibonacci   |
|--------------|-----------------|-----------------|-------------|
| insert       | $\Theta(\lg n)$ | $O(\lg n)$      | $\Theta(1)$ |
| find-min     | $\Theta(1)$     | $O(\lg n)$      | $\Theta(1)$ |
| extract-min  | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |
| union        | $\Theta(n)$     | $O(\lg n)$      | $\Theta(1)$ |
| decrease-key | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $\Theta(1)$ |
| delete       | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |

(Montículo de Fibonacci: complejidades **amortizadas**.)

## Comparando las tres estructuras



| Operación    | Binario         | Binomial        | Fibonacci   |
|--------------|-----------------|-----------------|-------------|
| insert       | $\Theta(\lg n)$ | $O(\lg n)$      | $\Theta(1)$ |
| find-min     | $\Theta(1)$     | $O(\lg n)$      | $\Theta(1)$ |
| extract-min  | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |
| union        | $\Theta(n)$     | $O(\lg n)$      | $\Theta(1)$ |
| decrease-key | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $\Theta(1)$ |
| delete       | $\Theta(\lg n)$ | $\Theta(\lg n)$ | $O(\lg n)$  |

(Montículo de Fibonacci: complejidades **amortizadas**.)

**El montículo de Fibonacci gana su ventaja siendo perezoso:** pospone el trabajo de reorganizar la estructura hasta EXTRACT-MIN.

## ¿Cuántos árboles hay en un montículo binomial?



### Observación estructural

Como hay a lo mucho un árbol  $B_k$  por cada orden  $k$ , y  $B_k$  tiene  $2^k$  nodos, un montículo binomial con  $n$  nodos tiene a lo mucho  $\lfloor \lg n \rfloor + 1$  árboles — tantos como bits en la representación binaria de  $n$ .

## ¿Cuántos árboles hay en un montículo binomial?



### Observación estructural

Como hay a lo mucho un árbol  $B_k$  por cada orden  $k$ , y  $B_k$  tiene  $2^k$  nodos, un montículo binomial con  $n$  nodos tiene a lo mucho  $\lfloor \lg n \rfloor + 1$  árboles — tantos como bits en la representación binaria de  $n$ .

### Consecuencia inmediata

FIND-MIN debe recorrer todas las raíces:  $O(\lg n)$ .

## Union: como sumar en binario



### La idea

1. Mezclar las dos listas de raíces (ordenadas por orden) en una sola lista ordenada:  $O(\lg n)$ , pues cada una tiene  $O(\lg n)$  árboles.
2. Recorrer la lista mezclada: si dos árboles **consecutivos** tienen el mismo orden  $k$ , enlazarlos en un  $B_{k+1}$  — igual que un *acarreo* al sumar en binario.

## Union: como sumar en binario



### La idea

1. Mezclar las dos listas de raíces (ordenadas por orden) en una sola lista ordenada:  $O(\lg n)$ , pues cada una tiene  $O(\lg n)$  árboles.
2. Recorrer la lista mezclada: si dos árboles **consecutivos** tienen el mismo orden  $k$ , enlazarlos en un  $B_{k+1}$  — igual que un *acarreo* al sumar en binario.

¿Por qué el recorrido es  $O(\lg n)$ ?

Hay a lo mucho  $O(\lg n)$  árboles en la lista mezclada, y cada posición se visita  $O(1)$  veces — igual que un acarreo no se propaga más de una vez por posición en una suma binaria normal.

## Binomial-Link: el paso elemental

---



---

### Algoritmo 4: BINOMIAL-LINK( $y, z$ )

---

```
//  $y, z$  son raíces del mismo orden  $k$ ; asumimos  $\text{llave}(y) \geq$   
   $\text{llave}(z)$   
 $\text{padre}(y) \leftarrow z$  ;  
 $\text{siguiente-hermano}(y) \leftarrow \text{hijo}(z)$  ;  
 $\text{hijo}(z) \leftarrow y$  ;  
 $\text{grado}(z) \leftarrow \text{grado}(z) + 1$  ;
```

---

## Binomial-Link: el paso elemental



---

### Algoritmo 5: BINOMIAL-LINK( $y, z$ )

---

```
//  $y, z$  son raíces del mismo orden  $k$ ; asumimos  $\text{llave}(y) \geq \text{llave}(z)$   
padre( $y$ )  $\leftarrow z$  ;  
siguiente-hermano( $y$ )  $\leftarrow$  hijo( $z$ ) ;  
hijo( $z$ )  $\leftarrow y$  ;  
grado( $z$ )  $\leftarrow$  grado( $z$ ) + 1 ;
```

---

$O(1)$ : solo se mueven punteros, nunca se copian nodos. Este es el “acarreo” que enlaza dos  $B_k$  en un  $B_{k+1}$ .

## Union: el algoritmo



---

### Algoritmo 6: $\text{UNION}(H_1, H_2)$

---

```
 $H \leftarrow$  mezclar las listas de raíces de  $H_1$  y  $H_2$ , ordenadas por grado ;  
 $x \leftarrow$  primera raíz de  $H$  ;  
mientras  $x$  tiene una siguiente raíz y con  $\text{grado}(x) = \text{grado}(y)$  hacer  
    si existe una tercera raíz  $z$  después de  $y$  con  $\text{grado}(z) = \text{grado}(x)$   
        entonces  
            avanzar  $x$  a  $y$  ;    // se enlazará en la siguiente vuelta  
    en otro caso  
        BINOMIAL-LINK( $x, y$ ) ;    // o al revés, según la llave  
            menor  
devolver  $H$  ;
```

---

## Union: el algoritmo



---

### Algoritmo 7: $\text{UNION}(H_1, H_2)$

---

$H \leftarrow$  mezclar las listas de raíces de  $H_1$  y  $H_2$ , ordenadas por grado ;  
 $x \leftarrow$  primera raíz de  $H$  ;  
**mientras**  $x$  tiene una siguiente raíz y con  $\text{grado}(x) = \text{grado}(y)$  **hacer**  
    **si** existe una tercera raíz  $z$  después de  $y$  con  $\text{grado}(z) = \text{grado}(x)$   
        **entonces**  
            avanzar  $x$  a  $y$  ;   // se enlazará en la siguiente vuelta  
    **en otro caso**  
        BINOMIAL-LINK( $x, y$ )   ;   // o al revés, según la llave  
            menor  
**devolver**  $H$  ;

---

INSERT( $H, x$ ) es apenas un caso particular:  $\text{UNION}(H, \{x\})$ , con  $x$  como un  $B_0$  nuevo.

## Bono: Insert es $O(1)$ amortizado

---



### La analogía con un contador binario

Insertar un nodo es como **incrementar en 1** un contador escrito en binario: si el bit menos significativo es 0, listo ( $O(1)$ ); si es 1, se propaga el acarreo (aquí, un enlace) hacia el siguiente orden, y así sucesivamente.

## Bono: Insert es $O(1)$ amortizado



### La analogía con un contador binario

Insertar un nodo es como **incrementar en 1** un contador escrito en binario: si el bit menos significativo es 0, listo ( $O(1)$ ); si es 1, se propaga el acarreo (aquí, un enlace) hacia el siguiente orden, y así sucesivamente.

### Análisis agregado

En  $n$  incrementos consecutivos de un contador de  $b$  bits, el bit  $i$  cambia a lo mucho  $\lceil n/2^i \rceil$  veces. Sumando sobre todos los bits: 
$$\sum_{i=0}^{b-1} \left\lceil \frac{n}{2^i} \right\rceil < 2n.$$

## Bono: Insert es $O(1)$ amortizado

---



### Conclusión

El número total de “acarreos” (enlaces) en  $n$  inserciones es  $O(n)$ : **amortizado**, cada INSERT cuesta  $O(1)$ , aunque un solo INSERT pueda costar  $O(\lg n)$  en el peor caso.

## Extract-Min: el algoritmo

---



---

### Algoritmo 8: EXTRACT-MIN( $H$ )

---

Encontrar la raíz  $x$  con menor llave, recorriendo la lista de raíces ;

Quitar  $x$  de la lista de raíces de  $H$  ;

$H' \leftarrow$  montículo binomial con los hijos de  $x$  como su lista de raíces ;

$H \leftarrow \text{UNION}(H, H')$  ;

**devolver**  $x$  ;

---

## Extract-Min: el algoritmo



---

### Algoritmo 9: EXTRACT-MIN( $H$ )

---

Encontrar la raíz  $x$  con menor llave, recorriendo la lista de raíces ;

Quitar  $x$  de la lista de raíces de  $H$  ;

$H' \leftarrow$  montículo binomial con los hijos de  $x$  como su lista de raíces ;

$H \leftarrow \text{UNION}(H, H')$  ;

**devolver**  $x$  ;

---

### Complejidad

Encontrar el mínimo recorre a lo mucho  $\lg n + 1$  raíces:  $O(\lg n)$ . Los hijos de  $x$  son a lo mucho  $\lg n$  raíces nuevas, y UNION cuesta  $O(\lg n)$ . Total:  $O(\lg n)$ .

## Decrease-Key y Delete

---

---

### Algoritmo 10: DECREASE-KEY( $H, x, k$ )

---

```
llave( $x$ )  $\leftarrow k$  ;  
 $y \leftarrow x$ ;  $z \leftarrow \text{padre}(y)$  ;  
mientras  $z \neq \text{nulo}$  y  $\text{llave}(y) < \text{llave}(z)$  hacer  
    | intercambiar  $\text{llave}(y)$  y  $\text{llave}(z)$  ;  
    |  $y \leftarrow z$ ;  $z \leftarrow \text{padre}(y)$  ;
```

---

## Decrease-Key y Delete



---

### Algoritmo 12: DECREASE-KEY( $H, x, k$ )

---

llave( $x$ )  $\leftarrow k$  ;  
 $y \leftarrow x$ ;  $z \leftarrow \text{padre}(y)$  ;  
**mientras**  $z \neq \text{nulo}$  **y**  $\text{llave}(y) < \text{llave}(z)$  **hacer**  
    intercambiar llave( $y$ ) y llave( $z$ ) ;  
     $y \leftarrow z$ ;  $z \leftarrow \text{padre}(y)$  ;

---

---

### Algoritmo 13: DELETE( $H, x$ )

---

DECREASE-KEY( $H, x, -\infty$ ) ;  
EXTRACT-MIN( $H$ ) ;

---

## Decrease-Key y Delete



---

### Algoritmo 14: DECREASE-KEY( $H, x, k$ )

---

```
llave( $x$ )  $\leftarrow k$  ;  
 $y \leftarrow x$ ;  $z \leftarrow \text{padre}(y)$  ;  
mientras  $z \neq \text{nulo}$  y  $\text{llave}(y) < \text{llave}(z)$  hacer  
    | intercambiar  $\text{llave}(y)$  y  $\text{llave}(z)$  ;  
    |  $y \leftarrow z$ ;  $z \leftarrow \text{padre}(y)$  ;
```

---

---

### Algoritmo 15: DELETE( $H, x$ )

---

```
DECREASE-KEY( $H, x, -\infty$ ) ;  
EXTRACT-MIN( $H$ ) ;
```

---

Un nodo en  $B_k$  está a profundidad a lo mucho  $k = O(\lg n)$  de su raíz: “burbujear” hacia arriba cuesta  $O(\lg n)$ , igual que en un montículo binario. DELETE también es  $O(\lg n)$ .

## Resumen de la sesión

---



## Resumen de la sesión

---



- ▶ El **montículo binario** es simple y  $\Theta(n)$  para construir, pero  $\Theta(n)$  para UNION.

## Resumen de la sesión

---



- ▶ El **montículo binario** es simple y  $\Theta(n)$  para construir, pero  $\Theta(n)$  para UNION.
- ▶ El **montículo binomial** arregla UNION ( $O(\lg n)$ ) organizando árboles como los bits de  $n$  en binario.
- ▶ En la próxima clase veremos — **Montículo de Fibonacci e implementación de heaps**

## Agradecimientos

---



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso, así como en clases similares de otras universidades.  
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz (<https://web.stanford.edu/class/archive/cs/cs166/cs166.1256/>).

Gracias